Implementing REST API

Lecture Notes

Topic: Implementing REST API Using Case Study

Duration: 2 Hours
Lecture Type: Theory + Step-by-Step Practical

📌 Learning Objectives

By the end of this lecture, students will be able to:

1. Understand what a REST API is.
2. Explain REST principles.
3. Design API endpoints.
4. Implement a REST API step-by-step.
5. Test API using tools like Postman.
6. Apply CRUD operations in real-life case studies.

🕑 Lecture Plan (2 Hours)

Time	Activity
0 – 20 min	Introduction to REST & APIs
20 – 40 min	REST Principles & HTTP Methods
40 – 60 min	Case Study Discussion + API Design
60 – 70 min	**Student Activity 1**
70 – 100 min	Step-by-Step Practical Implementation
100 – 110 min	Testing & Error Handling
110 – 120 min	**Student Quiz + Wrap-up**

Part 1: Introduction to API & REST (0 – 20 min)

What is an API?

API stands for **Application Programming Interface**.

👉 Real-life Example:
Think of a **restaurant waiter**:

- You (client) give order.
- Waiter (API) sends order to kitchen (server).
- Kitchen prepares food.
- Waiter brings food back to you.

So:

- Client = Frontend
- Server = Backend
- API = Communication bridge

What is REST?

REST stands for **Representational State Transfer**.

It is an architectural style for building web services.

It uses HTTP protocol.

Part 2: REST Principles (20 – 40 min)

REST is based on 6 main principles:

1️⃣ Client-Server Architecture

Frontend and backend are separate.

2️⃣ Stateless

Each request must contain all necessary information.
Server does NOT store client session.

Example:
Every time you go to ATM, you insert card again.

3️⃣ Cacheable

Responses can be cached to improve performance.

4️⃣ Uniform Interface

Standard URL and HTTP methods.

5️⃣ Layered System

Can have security layer, database layer etc.

6️⃣ Code on Demand (Optional)

HTTP Methods in REST

Method	Purpose	Example
GET	Retrieve data	Get student list
POST	Create new data	Add student
PUT	Update data	Update student
DELETE	Delete data	Remove student

🧠 Mini Discussion Question (After 30–40 Minutes)

Ask students:

1. Why REST APIs are stateless?
2. What problems occur if server stores session?
3. Can we use POST instead of GET for fetching data? Why/Why not?

Part 3: Case Study – Student Management System (40 – 60 min)

🎯 Problem Statement

We want to build a REST API for a **Student Management System**.

Each student has:

- id
- name
- email
- department

Step 1: Identify Resources

In REST, everything is a resource.

Resource here = **Student**

Base URL:

`/api/students`

Step 2: Design Endpoints

Operation	Method	URL
Get all students	GET	/api/students
Get student by id	GET	/api/students/{id}
Add student	POST	/api/students
Update student	PUT	/api/students/{id}
Delete student	DELETE	/api/students/{id}

👩‍🎓 Student Activity 1 (60 – 70 min)

Divide students into groups.

👉 Task:
Design REST API for:

- Library Management System OR
- Online Store

They must:

- Identify resources
- Define endpoints
- Choose HTTP methods

Discuss answers after 10 minutes.

Part 4: Step-by-Step Practical Implementation (70 – 100 min)

We will implement using:

- Node.js
- Express.js
- Postman

(You can also adapt to PHP/Laravel, Python/Django etc.)

Step 1: Initialize Project

```
npm init -y
npm install express
```

Create file:

```
server.js
```

Step 2: Basic Server Setup

```
const express = require('express');
const app = express();

app.use(express.json());

app.listen(3000, () => {
    console.log("Server running on port 3000");
});
```

Step 3: Create Dummy Data

```
let students = [
    { id: 1, name: "Ali", email: "ali@gmail.com", department: "CS" },
    { id: 2, name: "Sara", email: "sara@gmail.com", department: "IT" }
];
```

Step 4: Implement CRUD Operations

GET All Students

```
app.get('/api/students', (req, res) => {
    res.json(students);
});
```

GET Student By ID

```
app.get('/api/students/:id', (req, res) => {
    const student = students.find(s => s.id == req.params.id);
    if (!student) {
        return res.status(404).json({ message: "Student not found" });
    }
    res.json(student);
});
```

POST Create Student

```
app.post('/api/students', (req, res) => {
    const newStudent = {
        id: students.length + 1,
        ...req.body
    };
    students.push(newStudent);
    res.status(201).json(newStudent);
});
```

PUT Update Student

```
app.put('/api/students/:id', (req, res) => {
    const student = students.find(s => s.id == req.params.id);
    if (!student) {
        return res.status(404).json({ message: "Student not found" });
    }

    student.name = req.body.name;
    student.email = req.body.email;
    student.department = req.body.department;

    res.json(student);
});
```

DELETE Student

```
app.delete('/api/students/:id', (req, res) => {
    students = students.filter(s => s.id != req.params.id);
    res.json({ message: "Student deleted successfully" });
});
```

Part 5: Testing Using Postman (100 – 110 min)

Tool: Postman

Steps:

1. Open Postman
2. Select HTTP method
3. Enter URL
4. Send request
5. Check response

Common HTTP Status Codes

Code	Meaning
200	OK
201	Created
400	Bad Request
404	Not Found
500	Server Error

🧠 Final Student Quiz (110 – 120 min)

1. What does REST stand for?
2. Difference between PUT and POST?

3. Why APIs should be stateless?
4. Design endpoints for Teacher resource.
5. What status code is used when resource is created?

📝 Summary

Today we learned:

- What REST API is
- REST principles
- HTTP methods
- How to design endpoints
- How to implement CRUD
- How to test API

💡 Homework

1. Connect this API to MongoDB.
2. Add validation.
3. Implement authentication using JWT.
4. Deploy on cloud (e.g., Render or Railway).
